TECHNICAL REPORT / CPU BACKENDS

Compiling scheduled egglog rules into reusable CPU execution blocks

An architecture proposal for egglog and eggcc, evaluated on every workload in the passing suite at eggcc revision `ae80f82`.

Eggfuse compiles rule-local joins and effects into typed recipes, shares entry work across consumers, and carries checked references to committed constructor results across scheduled Runs. It leaves Free Join responsible for residual matches. The architectural proposal is to expose these operations as optional physical plans inside egglog, where the native engine can retain ownership of scheduling, merges and extraction.

Frozen inference	Tasks	Cell faster	Geomean N/C	Total N/C
Pass 1	96	96	2.413×	2.306×
Pass 2	96	96	2.124×	1.960×
Pass 3	96	96	2.144×	2.520×
Three-pass sum	96	96	2.197×	2.195×

N/C is original unmarked egglog replay time divided by Eggfuse replay time. Ratios above 1 favor Eggfuse. All 288 inputs have three measured pairs. Appendix A plots every workload; Appendix B lists every pass.

Across 96 workloads, the sum of per-pass median inference times is 145.504 s for native egglog and 66.286 s for Cell. Raytrace contributes 34.5% of the time saved; excluding it gives a time-weighted ratio of 1.988×. A fixed sum-bits block catalog transfers to all targets without per-target block training. Each pass starts from eggcc's captured production input. Offline preparation adds 826.63 s across the suite. Section 6 separates inference, process time, preparation and memory costs.

> **The adoption decision.** Prototype shared entry packets and checked register joins beneath egglog's existing evaluator. The current standalone backend has a concrete sum-bits endpoint discrepancy even though projected table counts agree. Treat the timing differences as architecture evidence. Require the native observable-result and extraction gates in §7 before enabling an alternative backend for production.

Cell `27f098a2` · Native `62ae6baa` · Apple M3 Max · arm64 macOS 27.0.1 · Rust 1.94.0. The corpus has 96 Bril/Rust workloads and 288 pass inputs. Scope, individual timings, replay scripts, witness and input digests are included in the companion artifact.

1. Where the compiler meets the engine

Eggcc supplies a scheduled equality-saturation program and uses an extractor to choose a result for its next pass. Egglog supplies incremental relational evaluation and congruence closure. A useful extension point is the compiled rule: it already identifies variables, table calls, key columns, effect destinations and ruleset ownership. Eggfuse makes those relationships explicit for a standalone executor. eggcc source · egglog semantics

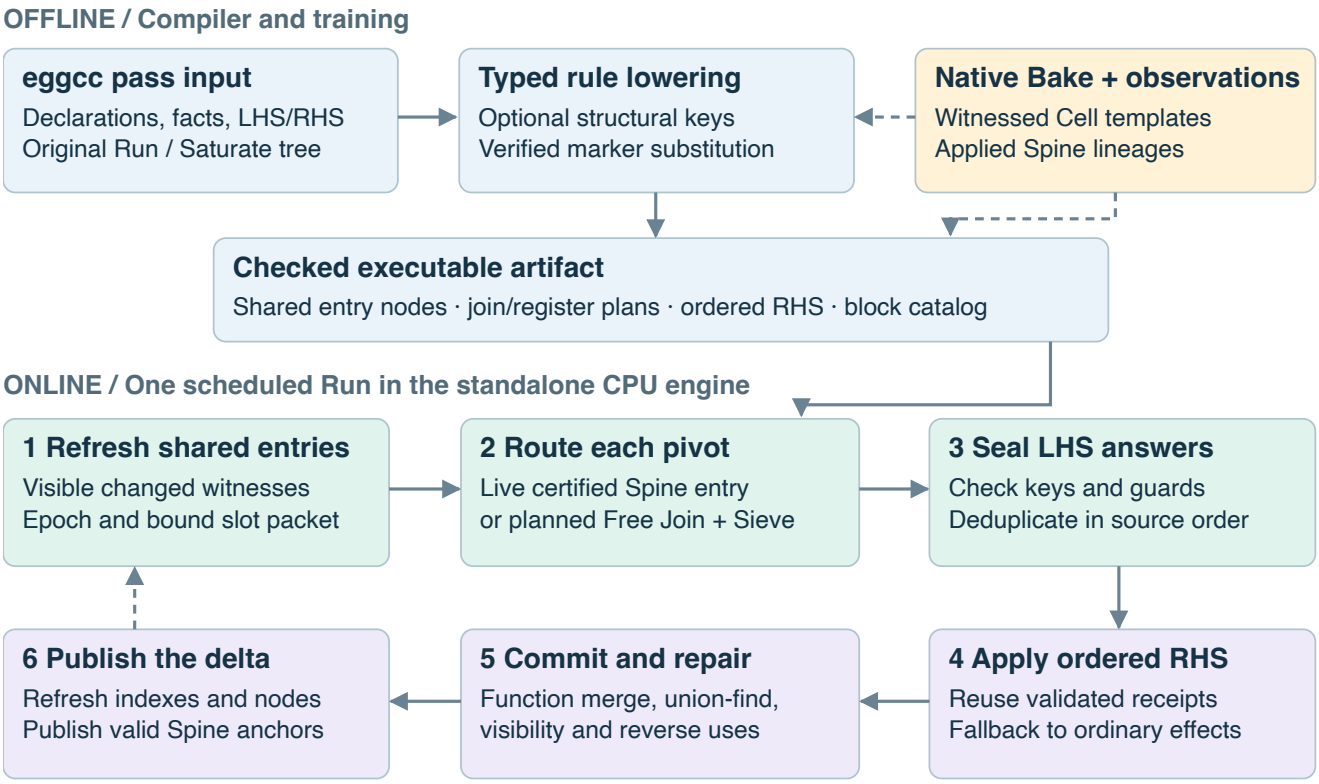


Fig. 1 The compiler produces a checked artifact. The colored runtime path repeats at the caller's next legal Run. Dashed inputs carry training evidence or refreshed state across a boundary. Native extraction sits outside the replay timer.

Two kinds of prior knowledge. The compiler proves facts such as "this key is bound" or "this RHS requests the same constructor as that producer." Training observes how often a path actually wins. The first supplies the legality conditions. The second decides whether spending memory and compilation effort on the path is worthwhile. Compiler entry · Artifact loader

What sharing can save. Suppose d changed rows satisfy one atom shape watched by m consumers. If all m separately decode and test that atom, its entry work is dmc, where c is the cost per decode and test. With a shared entry it is approximately $dc + d\sum_i p_i$, where p_i is the cost of projecting and delivering the packet to consumer i. Each consumer still pays for its residual join, sorting and effects. This local model predicts a saving only when dispatch and maintenance cost less than the redundant entry work. Egglog can already share physical indexes, so that common cost is outside this comparison.

2. From a rule to a reusable plan

SOURCE TRANSFORMATION

The Rust front end reads the flattened pass, installs explicitly requested structural-key selection, runs Bake on a compatible training input, selects markers, and lowers the transformed program to Recipe IR. A marker represents a positive conjunction with explicit ports. The compiler emits maintenance rules to populate it before dependent consumers run. Its selection problem trades saved consumer atoms against maintenance cost, with structural estimates or measured inputs. Observing a Cell during Bake proposes reuse. The source transformation still needs a rule-derived positive view and a legal place in the schedule. Marker selection · Cell witnesses

PHYSICAL LOWERING

A rule becomes typed query atoms, input slots, effect destinations and a source-owned control tree. The compiler enumerates legal pivot-specific orders. A register kernel can admit a connected query with one indexed expansion, subsequent full-key reads and primitives whose operands are bound. Fresh columns write registers. Repeated variables, literals and previously assigned columns generate equality checks. The loader reconstructs physical plans from the typed recipe and checks serialized plans. Kernel derivation · Kernel interpreter

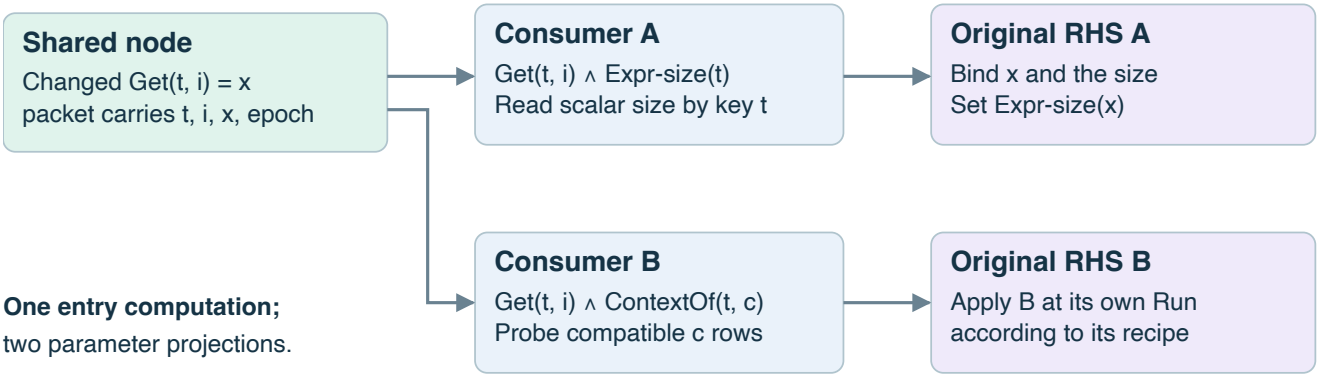


Fig. 2 Two consumers can subscribe to the same normalized atom shape. An arrival supplies a correlated row, its witness and epoch. Consumer projections map packet positions to local slots. An update to the other atom can also be the rule's pivot.

For a changed row r and consumer watermark w, entry selection restricts the delta to visible rows with $r.\text{epoch} \geq w$. The Sieve can prove that necessary domains do not intersect. Planned Free Join can probe compatible keys before materializing seeds and can start at a selective indexed domain. A fingerprint narrows the candidates. The matcher checks full values before emitting a binding. Ownership by epoch, witness and pivot plus source-slot deduplication eliminates duplicate derivations at the Run boundary. Shared nodes · Join routing · Sieve

3. Cells, repair and the Run boundary

A **Cell template** contains an ordered effect recipe. An **instance** records its template ID, bound slots, optional parent and live fact positions. A constructor fact can be named by a `FactRef {cell, op}`. The recipe tells the engine which relation and slot operands that position denotes. Decoding those operands produces the current logical row. A mutable function has a current key, value, owner and epoch. Query indexes and table widths still matter: storing facts through shared recipes trades duplicate representation for decoding, liveness tests and reference maintenance. Graph and repair · Frame storage

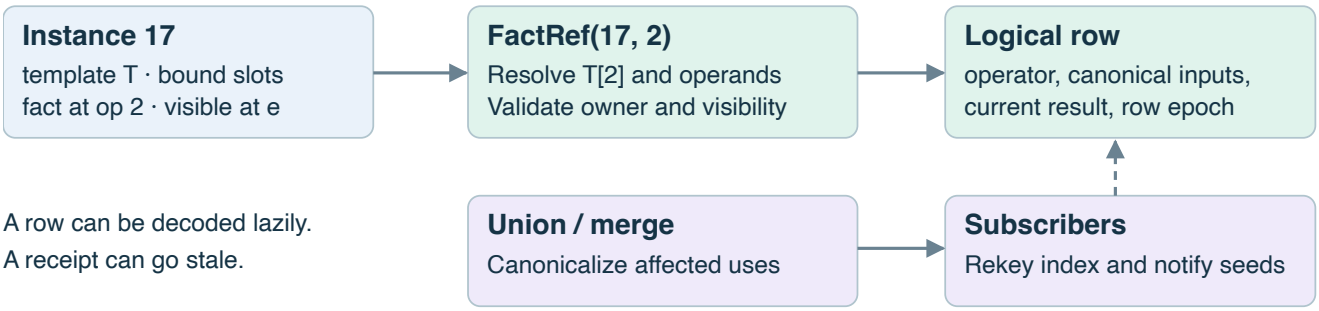


Fig. 3 The address of a constructor row combines a concrete Cell instance and an instruction position. A mutable function row has its own current owner. Purple shows the repair path after a key changes.

Physical object	What consumers may reuse	What can invalidate it
Cell fact position	A committed constructor witness	Deletion, subsumption, movement or a changed canonical key
Function receipt	The current committed function row	An ordered merge or replacement of its current value
Index bucket / packet	Candidate witnesses for a key and epoch	Membership, visibility or representative change
Spine cursor	Bounded producer lineage at a live anchor	A stale receipt or failed consumer precondition

EXECUTION CONTRACT

```
for each Run selected by the source schedule:
    read all rule LHSs against the current snapshot
    seal complete bindings in the rule's original input order
    apply their RHS instructions in order
    repair congruence and rekey affected rows
    publish visible changes, update subscribers and watermarks
    evaluate the next repeat/saturation condition
```

On union, reverse dependencies identify constructor and function keys involving an affected representative. The engine invalidates decoded payloads, handles rekeying or coalescence and delivers changes to the relevant indexes. Consumers must be able to observe old-key removal, new-key arrival, visibility and forwarding. A compiled block does not remove this repair obligation. A cheap RHS path can avoid registration of a temporary frame when a typed single function write can be completed directly. A failure of its preconditions resumes ordinary effects. Run and RHS execution · Storage and kernel ablation

4. How a block crosses Runs

A **Spine** follows committed facts that serve as anchors for subsequent rules. The compiler records which consumer pivot can read a producer's output and whether its remaining ports can be obtained exhaustively. The producer publishes a weak reference when its fact becomes visible. A later consumer can use the reference only at its own scheduled occurrence. The next matched rule can have other LHS entries, other matches and ordinary effects in the same Run.

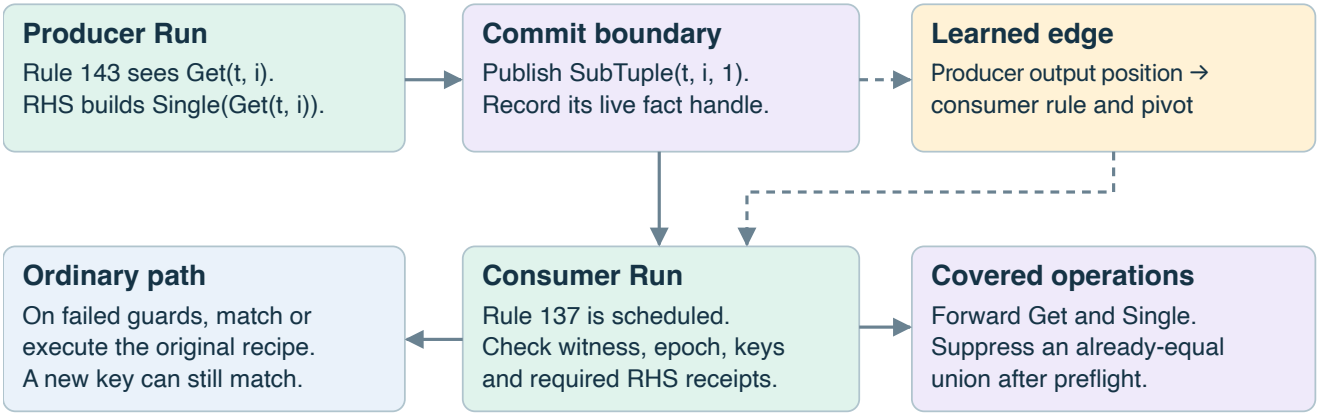


Fig. 4 An observed producer/consumer pair from Raytrace. The published SubTuple fact enables a later pivot. Producer equalities and constructor receipts can also satisfy part of the consumer RHS. The dashed edge requests a switch. The current runtime state decides whether it succeeds.

Concrete saving. Rule 143 witnesses `Get(t,i)`, constructs its `Single` and equates that result with `SubTuple(t,i,1)`. A later rule 137 requires the same constructor key. The compiler records the donor instruction, distance, and consumer destination. The runtime validates the current row and canonical keys before reusing it. A whole-RHS preflight checks every required receipt before any effect is committed. An unexpected replacement chooses the ordinary arm. This avoids failed speculation partially applying a write. Switch construction · Original pair study

Learning larger blocks. Start with 32 applied consumer/pivot leaves. A training round counts witnessed adjacent units on a winning lineage. A proposed binary unit names a left unit, the publishing instruction and a right unit. Freeze the catalog during each run, bound its size, and let an actually applied macro become a later operand. The donor used here has a 64-unit ceiling, at most four primitive steps, eight nodes, up to eight additions per round and a 512-entry candidate budget. When transferring, the harness asks the compiler to resolve each leaf's typed rule signature on the target. It remaps only reachable joins and rebuilds every RHS handoff against that target. Typed leaf transfer · Bounded composition

5. Performance against native egglog

This study replays all 96 passing workloads at eggcc revision `ae80f82`. Each of the three captured pass inputs is run in a fresh native process and a fresh Cell process three times. Per-pass values are medians. A three-pass workload value sums those three medians. The native runner executes the original unmarked flat program; Cell compiles that input and links the same donor catalog through checked rule signatures.

Frozen inference	Workloads	Cell faster	Geomean N/C	Total N/C
Pass 1	96	96	2.413×	2.306×
Pass 2	96	96	2.124×	1.960×
Pass 3	96	96	2.144×	2.520×
Three-pass sum	96	96	2.197×	2.195×

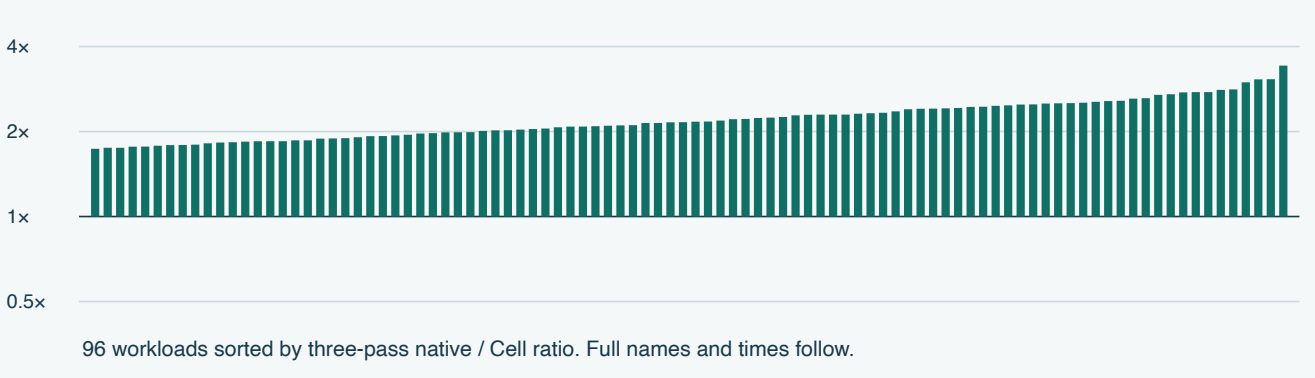


Fig. 5 One bar per workload, sorted by the ratio of native egglog to Cell time for three frozen pass inputs. Values above 1 favor Cell. Appendix A labels every workload and gives both totals; Appendix B gives all per-pass medians.

Across the corpus, native inference totals 145.504 s and Cell inference totals 66.286 s. Cell is faster on 96 of 96 three-pass totals and slower on 0. The equal-weight geometric mean is 2.197×. Raytrace contributes 34.5% of the time saved; excluding it gives a time-weighted ratio of 1.988×.

REPEATED PASS 3 ANCHORS

Workload	Pairs	Native ms	Cell ms	N/C	Cell faster
sum-bits	3	44.134	18.181	2.428×	3
fenwick_tree	3	108.046	43.940	2.459×	3
raytrace	3	23,790.953	6,966.665	3.415×	3

These anchors are drawn from the same repeated full-suite study. They use ordinary release builds with one thread. Offline compilation and linking are recorded separately. The comparison combines the checked compiler representation with its execution engine, so an upstream trial should isolate the matcher, action and storage interfaces before adopting them.

> **Interpretation.** Both sides receive the same captured pass boundary. The refreshed Cell endpoints match the archived Cell endpoints in 288 of 288 inputs. Section 7 documents the independent native/Cell membership discrepancy. The timing evidence motivates an architecture experiment; the native-observable correctness gate still applies.

6. A matched replay protocol

INPUTS AND EXECUTABLES

Eggcc's captured production run used revision `ae80f82` and its pinned `egglog-experimental 08771f9`, with Tiger extraction supplying the next pass. The supplied archive contains the resulting 288 flat pass inputs. This experiment freezes those boundaries. It does not substitute a new extractor into the production chain.

The native comparator is the immutable `egglog-baseline` snapshot `62ae6baa`. Its Cargo-built helper sets one worker and times execution from the first `RunSchedule` onward. Native search/merge counter hooks are compiled out. The Cell compiler and executor use source `27f098a2`, ordinary release settings and the sparse storage/shared-view execution features. Neither arm uses PGO or a machine-specific code-generation override in this suite.

COMPILER TREATMENT

Every Cell target receives the same small marker-training source and the same iterative donor learned from real sum-bits Pass 3 executions. Sum-bits is the donor workload; the other workloads test transfer. Marker training and block composition supply different information. The linker resolves typed rule signatures, retains reachable joins and rebuilds RHS handoff proofs against each target. The catalog contains 59 deployable units on Pass 1/2 and 64 on Pass 3. There is no per-target block training or per-workload runtime exception.

TIMING AND OUTPUT

For each input the harness compiles and links once, then alternates three fresh native/Cell process pairs. It records all samples, failures, source hashes, deployed units, Cell Run counts and endpoint hashes. The pass replay order is P3, P1, P2. The workstation is unisolated, so close results need additional repetitions.

Inference begins after input loading and initialization. Cell's interval includes scheduled search, RHS, repair and publication. Complete CLI adds loading and output; Cell also exports the full graph. Native RSS covers its whole process without graph export, and Cell RSS is sampled immediately after inference, before export. The process-level columns therefore describe each executable's actual obligations.

Pass 3	Native RSS MiB	Cell RSS MiB	Native CLI ms	Cell CLI ms
sum-bits	36.20	168.73	109.91	117.74
fenwick_tree	42.89	189.56	173.36	154.70
raytrace	470.78	1,855.33	23,955.09	14,801.89

Across all 288 inputs, summed CLI medians are 165.828 s for native and 141.316 s for Cell, a 1.173× ratio before offline preparation. This process-level comparison includes their different output duties and is distinct from the inference ratio.

PREPARATION AND AMORTIZATION

Preparation totaled 826.63 s across the 288 Cell artifacts. The runner repeats the shared Bake and lowering for each target; caching this preparation is a future integration choice. Comparing this cost with the 79.218 s aggregate inference saving gives an illustrative break-even of 10.4 replays of the entire prepared suite, assuming unchanged startup and export costs. One compilation per one execution has a different end-to-end trade-off.

The hashes and raw repetitions are in `suite-current.jsonl` and `evidence/suite-current-provenance.json`. `reproduce/refresh-suite.sh` builds all four executables with Cargo; its Python driver orchestrates the measurements.

7. Semantic boundary and upstream decision

Archived within-Cell audits compare ordered RHS inputs, schedules, committed effects, final graphs and winning Spine lineages. Sum-bits has 1,428 audited Runs, 3,034 resumed steps and maximum depth 12; Raytrace has 16,883 Runs, 663,403 steps and depth 29. All 288 refreshed Cell endpoints match their archived references. Repetitions also agree on schedule hashes, bindings and resumed steps. These checks do not establish equivalence with egglog.

> **A reproducible difference.** On sum-bits, both the original native export and current fixed-64 Cell export have 3,206 rows after projection to original table names; all per-table counts agree. Grounded reconstruction nevertheless finds a visible Cell `TermSingle` with no matching native key. All 19 constructor occurrences in the witness resolve to visible Cell rows. Appending the ground check to the original flat source gives `CheckError` in the timed baseline binary. The same check fails on the marked input in native egglog. The archive includes the concrete term, both exports and replay commands.

The diverging term can be written compactly with `g(i) = TermGet(TermArg(), i)` and `b(op,x,y) = TermBop(op(),x,y)`:

```
q = b(Div, g(1), g(2))
p = b(Mul, q, TermConst(Int(2)))
witness = TermSingle(b(Add, g(0), b(Sub, g(1), p)))
```

This locates an observable membership difference. It leaves open whether the changed graph produces a different Tiger-extracted program and which implementation decision first caused the divergence. In particular, an order-sensitive merge can depend on an earlier schedule or binding-order difference. The table below identifies a smaller upstream experiment that would distinguish these causes.

Proposed seam	Retain in egglog	Acceptance evidence
Compiled LHS entry	Native Run snapshot, watermark and action-buffer order	Shadow each pivot and compare complete substitutions, including misses and ties
Register / writer plans	Native value interning, conflict resolution and application order	Compare key reads and ordered effects for insert, replace and no-op
Receipt publication	Native rebuild, subsumption and canonicalization	Invalidate on old/new keys, visibility and forwarding; test deleted and merged witnesses
Cross-Run block	Native scheduling and extraction	Check guards against current rows. Compare grounded queries and extracted IR through all passes

The first implementation milestone is an opt-in alternative matcher feeding egglog's ordinary action buffer. Run the exact counterexample as a regression and capture the first divergent source Run. Only after equality partitions, function outputs, visibility and selected terms agree should a production trial replace native effect or storage handling. A final trial should include offline cost amortization, total compiler wall time, peak memory and execution of the resulting binaries.

8. Distribution and upstream adoption

Workload	Native total ms	Cell total ms	Native / Cell
bril/float/norm.bril	384.01	220.96	1.738×
bril/mixed/cholesky_decomp.bril	1,354.18	772.89	1.752×
polybench/stencils/jacobi-1d.rs	778.24	444.15	1.752×
polybench/linear-algebra/solvers/lu.bril	1,812.28	1,025.07	1.768×
bril/core/fitsinside.bril	29.16	8.52	3.424×
bril/core/fact.bril	53.11	17.33	3.065×
bril/core/recfact.bril	54.09	17.67	3.062×
raytrace/raytrace.rs	41,057.38	13,741.25	2.988×

The first four entries have the smallest ratios and the last four the largest. Every workload contributes equally to the geometric mean, while total time weights longer programs more. Appendix A shows all 96 cases on a common log-ratio scale and gives exact totals. Appendix B shows where each result arises across the three passes.

KEEP EGGLOG'S SEMANTIC OWNERSHIP

Native egglog already provides Free Join, incremental indexes, vectorized action buffers and merge/rebuild machinery. The proposed integration adds compiler-proved shared entry packets and references to committed producer results at those interfaces. An upstream implementation can preserve native scheduling, interning, merge semantics and extraction while independently selecting the physical plan.

An entry compiler can share a normalized input across consumers, carry checked parameter projections, and discharge functional reads before a residual join. A successful match still enters the original ordered action buffer. A cross-Run continuation can reuse constructor results only after validating visibility and canonical keys at the consumer's scheduled occurrence.

THE MEMORY COST REMAINS PART OF THE PROPOSAL

The standalone representation also retains typed compilation metadata, producer slots, generated view/completion rows and auxiliary indexes. The native/Cell peak memory columns in Section 6 belong to this complete suite configuration. Section 9 measures a separate lifetime optimization on two inputs. Its lower RSS cannot be attached to the faster suite executable without another matched run.

A native integration should first test whether the existing native row storage can own the shared entry and completion metadata. The evidence favors measuring the amount of shared work and its lifecycle cost together. It leaves the grounded semantic gate in Section 7, extracted-program behavior, and input-to-output compiler wall time as explicit acceptance requirements.

> **A reviewer can inspect every case.** Appendix A labels all workload ratios and includes their native/Cell totals. The per-pass table and CSV preserve every case and its source identity. The raw JSONL records all three pairs for every input.

9. Memory and cross-Run lifetime

6 NOVEMBER 2026 / TWO-WORKLOAD MEMORY STUDY

This follow-up measures the standalone Cell CPU engine at `27f098a2` and the lifetime changes implemented in `4d9ea942`. It uses the frozen sum-bits and Raytrace Pass 3 artifacts, their original 64-unit Bake catalog, one CPU thread and ordinary release builds. The memory matrix has its own control and ordinary release configuration; no PGO is used here.

Pass 3	Control ms	Reclaimed ms	Control MiB	Reclaimed MiB
sum-bits	19.158	18.740	152.18	146.81
raytrace	6,746.279	6,716.853	1,856.79	1,432.68

Raytrace's peak RSS fell by **22.8%**. In the same matrix, complete CLI medians were 10.373 and 10.374 s. The median of paired inference differences was +16.663 ms, with only one of four reclaimed runs faster. A lower ratio of medians therefore supplies little evidence of a latency improvement.

RECHECKING THE NATIVE MEMORY GAP

Two new runs of eggcc's pinned native egglog `ebba7bb` executed the original unmarked Raytrace source with a verified one-worker pool. macOS `time -l` recorded 462.375 and 450.047 MiB peak RSS; the median was 456.21 MiB. Native timing included parsing, initialization and inference, without a graph export. Cell RSS was sampled after inference and before its export, including its artifact loader and generated tables. The control/native memory ratio was approximately **4.07×**, falling to **3.14×** with reclamation. Source and binary hashes accompany the measurements.

COUNT LIVE ALLOCATIONS ONCE

A diagnostic allocator attributes requested bytes to the allocating operation and subtracts them on deallocation. A shared Rc allocation is charged once. Reallocation retains its original category. At Raytrace inference completion it counted **1,762.98 MiB** still live.

Allocation origin	Live requested MiB
graph	494.6
artifact	378.5
search	215.4
columns	163.4
entries	149.0
spine	121.3
other	77.0
row payload	76.9

The largest remaining categories include RHS (62.6 MiB), Sieve (22.1 MiB) and captured history (2.3 MiB). Allocation origin can differ from the eventual owner: matched parameters can become persistent Cell slots. Category peaks occur at different times. The ledger excludes allocator fragmentation and C allocations; its instrumented timing and RSS are excluded from the speed comparison.

Evidence: `evidence/memory-lifetimes.json`, rendered by `reproduce/render-memory-data.py`. Program identities and all three memory matrices are recorded in the evidence file.

Consumers determine retention

A shared entry node owns a set of declared rule consumers. After a source ruleset occurrence, each rule's logical watermark advances, including rules omitted by the ready set. The node can release a materialized packet once its epoch is **strictly below every consumer's watermark**. An equal epoch can still seed a match. A rule appearing in several sets advances with its latest actual occurrence.

The implementation keeps a compact membership header so join planning sees the same cardinality. A changed fact can rebuild its packet through the existing publication stream. An explicitly older watermark request marks the node cold and reconstructs its visible rows. Reclamation can run immediately or conservatively every 32 graph epochs. The latter delays release without advancing the readers.

On Raytrace, immediate reclamation reduced retained entry packets from **665,584** to **82,355**. Entry-origin requested storage fell from 149.0 to 72.4 MiB. There were 882,328 retirements over the schedule, including repeated incarnations of some witnesses.

COMPLETION RECEIPTS NEED FEWER PRODUCER OWNERS

The final Raytrace Cell graph has 1,034,514 logical rows. Generated `CellSpineView*` tables account for 213,334; `CellSpineApplied*` tables account for 121,502. Together they contribute **32.4**% of the rows. This table-name grouping describes the measurement. The optimization uses executable read contracts.

The three populated Applied tables have no declared query or RHS readers. Their Unit values use an old-value merge. The compiler derives origin demand from all rule queries, executable table reads and calls in merge expressions. It checks an emitted certificate against the target program. An unread receipt can keep its key, value and rebuild identity while releasing its optional creating-frame reference. An assertion or later declared rule that reads the table retains that producer.

Pass 3	Full live Cells	Embedded snapshots	Both forms
sum-bits	2,385 → 2,177	490 → 639	2,875 → 2,816
raytrace	601,924 → 485,196	85,989 → 163,541	687,913 → 648,737

Some remaining producers change from full Cells into embedded snapshots after an unnecessary second owner is removed. The combined count measures that distinction. Every completion-table row stays in the logical export. A caller choosing the new mode can receive `None` from the optional origin accessor for an unread receipt.

RELEASE VALIDATION IR AFTER INITIALIZATION

A teardown probe freed about 323.7 MiB by dropping the original Raytrace validation JSON, plus 17.1 MiB for its input buffer. The explicit `--release-source-artifact` option releases these after runtime plans are initialized. Typed plans, structural-key nodes and the original source SHA remain available. The residual object has a provenance-only schema, which the source loader rejects. Offline Bake or revalidation requires reloading the original artifact.

For sum-bits, the combined live-requested footprint fell from 131.99 to 22.08 MiB, while peak RSS changed by much less. Its parser/setup high-water had already occurred. This separates the benefit of reclaiming completed input state from the cost of initially loading it.

Cost, correctness and remaining scope

The primary Raytrace matrix isolates entry retirement from validation-IR release.

Arm	Inference s	Peak RSS MiB
Control	6.822	1,864.23
Consumed entries	6.813	1,788.11
Validation IR	6.816	1,549.02
Both	6.719	1,475.88

The final matrix combines all three changes, with 32-epoch retirement. It rotates eight processes per sum-bits arm and four per Raytrace arm. Compilation and diagnostic runs are outside the measurements.

Pass 3	Paired delta	Faster pairs	Control CLI ms	Reclaimed CLI ms
sum-bits	-0.362 ms	6/8	116.665	116.369
raytrace	+16.663 ms	1/4	10,372.988	10,374.331

The primary sum-bits retirement-only arm was slower in all eight pairs. The evidence supports a memory reduction with a small, workload-dependent maintenance cost. Both lifetime features remain opt-in.

AUDIT THE SAME SCHEDULED OBSERVATIONS

Full audits matched the frozen graph, schedule, ordered RHS inputs, effective writes and Spine trajectory. They include the combined mode and forced fingerprint/Sieve collisions. Twenty-eight runtime tests and fifteen compiler tests passed, covering reader lifetimes, backward watermark requests and rejected certificates.

These are within-Cell checks. The grounded cross-engine witness in §7 requires its own resolution. The memory comparison establishes within-Cell equivalence. Sections 5–6 give the separate 96-workload native/Cell replay comparison.

WHAT CROSS-RUN COMPLETION CAN STILL ABSORB

History-reference trimming can recover only a few MiB in this measurement. Larger structures hold graph records, parameters, indexes and shared entry state. Future blocks should expose reader demand and last-use ports, then measure reconstruction costs against another retained row copy.

Generated view and completion rows are the next candidate. A compressed form must still support required canonical-key lookups, union forwarding, last-writer publication and exports. Their generated origin alone cannot justify removal.

> **Reproduce the update.** `reproduce/memory-replay.sh` accepts checkout `4d9ea942` and two frozen program files. Features `entry-retirement` and `origin-demand` select lifetime optimizations; `--release-source-artifact` releases validation IR. Bundled data contains all three matrices and their hashes.

APPENDIX A / NATIVE EGGLOG VERSUS EGGFUSE

Every workload · 1 / 4

Three-pass frozen-input inference, ordered by native / Cell time. Each pass uses the median of three fresh processes. All four pages share the same log-ratio scale. A point right of 1× favors EggFuse. LA abbreviates *linear-algebra*.

Workload (ID links to per-pass table)	Native / Cell (0.5× 1× 2× 4×)	Native ms	Cell ms	Ratio
045 bril/float/norm.bril		384.01	220.96	**1.738×**
062 bril/mixed/cholesky_decomp.bril		1,354.18	772.89	**1.752×**
092 polybench/stencils/jacobi-1d.rs		778.24	444.15	**1.752×**
083 polybench/LA/solvers/lu.bril		1,812.28	1,025.07	**1.768×**
090 polybench/stencils/fdtd-2d.rs		5,958.29	3,368.02	**1.769×**
047 bril/float/riemann.bril		152.22	85.43	**1.782×**
050 bril/mem/adj2csr.bril		601.82	336.13	**1.790×**
038 bril/float/conjugate-gradient.bril		1,272.93	710.05	**1.793×**
075 polybench/LA/kernels/3mm.bril		5,542.36	3,083.76	**1.797×**
087 polybench/medley/floyd-warshall.rs		1,383.76	761.59	**1.817×**
005 bril/core/bitwise-ops.bril		583.42	319.11	**1.828×**
076 polybench/LA/kernels/atax.bril		1,523.27	831.05	**1.833×**
023 bril/core/palindrome.bril		378.78	205.64	**1.842×**
074 polybench/LA/kernels/2mm.bril		4,363.72	2,361.69	**1.848×**
067 polybench/LA/blas/gemm.bril		2,560.55	1,385.39	**1.848×**
084 polybench/LA/solvers/ludcmp.bril		3,335.84	1,804.77	**1.848×**
022 bril/core/orders.bril		206.81	111.12	**1.861×**
066 polybench/datamining/covariance.rs		3,306.14	1,776.09	**1.861×**
072 polybench/LA/blas/syrk.bril		1,594.27	844.55	**1.888×**
068 polybench/LA/blas/gemver.bril		3,538.39	1,871.61	**1.891×**
080 polybench/LA/solvers/cholesky.bril		2,121.52	1,119.72	**1.895×**
065 polybench/datamining/correlation.rs		4,778.88	2,503.06	**1.909×**
085 polybench/LA/solvers/trisolv.bril		815.00	423.17	**1.926×**
081 polybench/LA/solvers/durbin.bril		1,001.21	519.31	**1.928×**

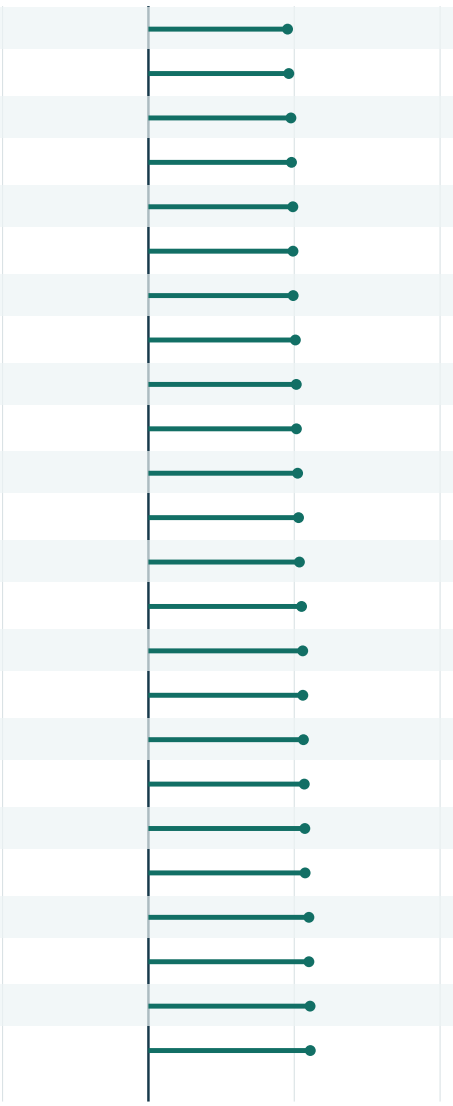
Three-pass totals sum the median of 3 fresh processes per pass. Right of 1× favors Cell.

APPENDIX A / NATIVE EGGLOG VERSUS EGGFUSE

Every workload · 2 / 4

Three-pass frozen-input inference, ordered by native / Cell time. Each pass uses the median of three fresh processes. All four pages share the same log-ratio scale. A point right of 1× favors EggFuse. LA abbreviates *linear-algebra*.

Workload (ID links to per-pass table)	Native / Cell (0.5× 1× 2× 4×)	Native ms	Cell ms	Ratio
064 fenwick/fenwick_tree.rs		447.24	230.84	**1.937×**
042 bril/float/mandelbrot.bril		480.88	246.78	**1.949×**
078 polybench/LA/kernels/doitgen.bril		2,371.58	1,204.69	**1.969×**
088 polybench/medley/nussinov.rs		2,001.62	1,013.97	**1.974×**
057 bril/mem/mat-mul.bril		544.45	274.06	**1.987×**
073 polybench/LA/blas/trmm.bril		1,362.32	685.21	**1.988×**
059 bril/mem/quickselect.bril		439.32	220.78	**1.990×**
053 bril/mem/bubblesort.bril		364.50	181.28	**2.011×**
079 polybench/LA/kernels/mvt.bril		1,525.96	755.66	**2.019×**
086 polybench/medley/deriche.rs		8,950.49	4,429.36	**2.021×**
071 polybench/LA/blas/syr2k.bril		2,039.39	1,003.54	**2.032×**
060 bril/mem/quicksort.bril		669.23	327.87	**2.041×**
077 polybench/LA/kernels/bicg.bril		1,671.28	815.32	**2.050×**
043 bril/float/n_root.bril		239.38	115.60	**2.071×**
054 bril/mem/csrmv.bril		1,522.60	731.30	**2.082×**
034 bril/core/sum-divisors.bril		204.30	98.07	**2.083×**
082 polybench/LA/solvers/gramschmidt.bril		2,706.98	1,295.75	**2.089×**
027 bril/core/pythagorean_triple.bril		139.69	66.59	**2.098×**
026 bril/core/primes-between.bril		374.40	178.02	**2.103×**
093 polybench/stencils/jacobi-2d.rs		2,543.82	1,207.80	**2.106×**
069 polybench/LA/blas/gesummv.bril		1,258.64	586.99	**2.144×**
089 polybench/stencils/adi.rs		4,161.26	1,940.64	**2.144×**
051 bril/mem/adler32.bril		232.33	107.75	**2.156×**
037 bril/core/up-arrow.bril		237.47	110.01	**2.159×**



Three-pass totals sum the median of 3 fresh processes per pass. Right of 1× favors Cell.

APPENDIX A / NATIVE EGGLOG VERSUS EGGFUSE

Every workload · 3 / 4

Three-pass frozen-input inference, ordered by native / Cell time. Each pass uses the median of three fresh processes. All four pages share the same log-ratio scale. A point right of 1× favors EggFuse. LA abbreviates *linear-algebra*.

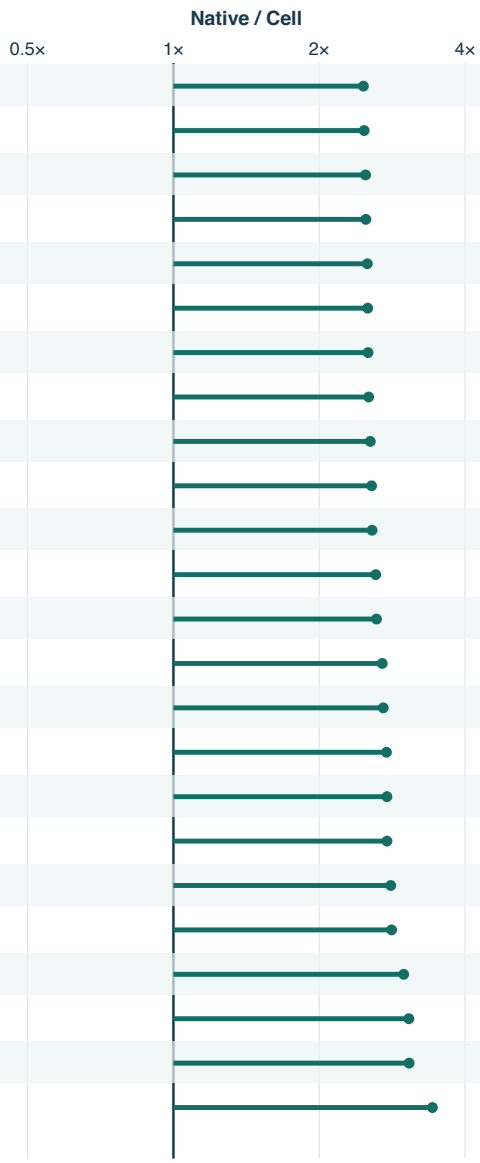
Workload (ID links to per-pass table)	Native / Cell (0.5× 1× 2× 4×)	Native ms	Cell ms	Ratio
000 bril/core/ackermann.bril		44.27	20.43	2.167×
039 bril/float/cordic.bril		1,681.78	774.85	2.170×
046 bril/float/ray-sphere-intersection.bril		54.96	25.13	2.187×
035 bril/core/sum-sq-diff.bril		141.00	63.74	2.212×
070 polybench/LA/blas/symm.bril		2,425.82	1,094.50	2.216×
094 polybench/stencils/seidel-2d.rs		1,840.48	824.17	2.233×
048 bril/float/sqrt.bril		174.70	77.98	2.240×
004 bril/core/bitshift.bril		65.57	29.11	2.252×
007 bril/core/check-primes.bril		193.54	84.77	2.283×
036 bril/core/totient.bril		253.02	110.33	2.293×
016 bril/core/gcd.bril		110.97	48.33	2.296×
009 bril/core/digital-root.bril		148.81	64.80	2.296×
052 bril/mem/binary-search.bril		198.09	86.25	2.297×
030 bril/core/relative-primes.bril		151.90	65.66	2.313×
031 bril/core/reverse.bril		181.12	77.98	2.323×
025 bril/core/perfect.bril		178.32	76.50	2.331×
008 bril/core/collatz.bril		115.80	49.12	2.358×
021 bril/core/mod_inv.bril		194.97	81.29	2.398×
061 bril/mem/vsmul.bril		185.23	76.90	2.409×
028 bril/core/quadratic.bril		134.60	55.82	2.411×
056 bril/mem/major-elm.bril		200.78	83.10	2.416×
063 bril/mixed/mat-inv.bril		432.18	178.30	2.424×
018 bril/core/is-decreasing.bril		123.36	50.45	2.445×
058 bril/mem/max-subarray.bril		263.90	107.76	2.449×

Three-pass totals sum the median of 3 fresh processes per pass. Right of 1× favors Cell.

APPENDIX A / NATIVE EGGLOG VERSUS EGGFUSE

Every workload · 4 / 4

Three-pass frozen-input inference, ordered by native / Cell time. Each pass uses the median of three fresh processes. All four pages share the same log-ratio scale. A point right of 1× favors EggFuse. LA abbreviates *linear-algebra*.



Workload (ID links to per-pass table)	Native / Cell	Native ms	Cell ms	Ratio
006 bril/core/catalan.bril		138.39	56.10	2.467×
091 polybench/stencils/heat-3d.rs		6,490.33	2,621.31	2.476×
001 bril/core/armstrong.bril		274.83	110.26	2.493×
044 bril/float/newton.bril		135.28	54.21	2.496×
013 bril/core/factors.bril		169.22	67.33	2.513×
041 bril/float/leibniz.bril		184.76	73.37	2.518×
015 bril/core/fizz-buzz.bril		180.72	71.66	2.522×
033 bril/core/sum-check.bril		132.94	52.54	2.530×
019 bril/core/lcm.bril		108.86	42.71	2.549×
055 bril/mem/fib.bril		172.70	67.34	2.564×
040 bril/float/euler.bril		90.61	35.25	2.570×
032 bril/core/sum-bits.bril		86.27	32.98	2.616×
017 bril/core/hanoi.bril		96.43	36.73	2.625×
010 bril/core/dot-product.bril		82.98	30.76	2.698×
024 bril/core/pascals-row.bril		132.02	48.70	2.711×
002 bril/core/binary-fmt.bril		66.18	24.04	2.752×
003 bril/core/birthday.bril		100.33	36.37	2.759×
049 bril/long/dead-branch.bril		136.28	49.39	2.759×
020 bril/core/loopfact.bril		88.82	31.61	2.809×
011 bril/core/euclid.bril		109.75	38.91	2.820×
095 raytrace/raytrace.rs		41,057.38	13,741.25	2.988×
029 bril/core/recfact.bril		54.09	17.67	3.062×
012 bril/core/fact.bril		53.11	17.33	3.065×
014 bril/core/fitsinside.bril		29.16	8.52	3.424×

Three-pass totals sum the median of 3 fresh processes per pass. Right of 1× favors Cell.

APPENDIX B / FROZEN REPLAY

Per-workload inference

Median milliseconds from three fresh processes per backend. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
0	bril/core/ackermann.bril	17.49	7.08	10.77	4.69	16.00	8.65	2.167×
1	bril/core/armstrong.bril	126.39	44.95	83.98	35.06	64.46	30.25	2.493×
2	bril/core/binary-fmt.bril	40.37	13.73	10.62	3.83	15.19	6.48	2.752×
3	bril/core/birthday.bril	41.75	15.30	45.34	16.64	13.24	4.43	2.759×
4	bril/core/bitshift.bril	28.39	10.33	11.61	5.07	25.56	13.71	2.252×
5	bril/core/bitwise-ops.bril	122.58	55.46	184.58	82.44	276.26	181.21	1.828×
6	bril/core/catalan.bril	37.35	13.42	61.16	24.75	39.88	17.93	2.467×
7	bril/core/check-primes.bril	85.36	37.78	63.65	25.78	44.53	21.21	2.283×
8	bril/core/collatz.bril	67.91	26.57	19.27	8.50	28.62	14.06	2.358×
9	bril/core/digital-root.bril	72.73	29.09	17.34	7.52	58.74	28.19	2.296×
10	bril/core/dot-product.bril	21.52	8.46	21.08	7.99	40.38	14.31	2.698×
11	bril/core/euclid.bril	31.23	10.03	54.21	20.80	24.30	8.09	2.820×
12	bril/core/fact.bril	34.97	11.14	7.79	2.46	10.35	3.72	3.065×
13	bril/core/factors.bril	64.45	24.03	71.47	29.59	33.31	13.71	2.513×
14	bril/core/fitsinside.bril	9.61	2.82	9.55	2.73	10.01	2.97	3.424×
15	bril/core/fizz-buzz.bril	84.59	31.71	61.29	25.06	34.85	14.89	2.522×
16	bril/core/gcd.bril	61.11	25.17	17.88	8.84	31.99	14.32	2.296×
17	bril/core/hanoi.bril	62.33	21.55	13.37	5.22	20.73	9.95	2.625×
18	bril/core/is-decreasing.bril	52.89	20.40	17.15	7.37	53.32	22.68	2.445×
19	bril/core/lcm.bril	50.69	18.75	14.73	5.80	43.44	18.16	2.549×
20	bril/core/loopfact.bril	38.90	14.50	38.78	13.44	11.14	3.68	2.809×
21	bril/core/mod_inv.bril	40.11	14.05	81.62	32.22	73.24	35.02	2.398×
22	bril/core/orders.bril	88.75	39.66	26.35	17.39	91.71	54.08	1.861×
23	bril/core/palindrome.bril	128.06	61.03	124.50	72.30	126.22	72.30	1.842×

APPENDIX B / FROZEN REPLAY

Per-workload inference

Median milliseconds from three fresh processes per backend. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
24	bril/core/pascals-row.bril	51.79	19.67	58.16	21.83	22.06	7.19	2.711×
25	bril/core/perfect.bril	95.35	37.62	21.13	9.11	61.84	29.77	2.331×
26	bril/core/primes-between.bril	135.47	63.56	118.39	50.99	120.53	63.47	2.103×
27	bril/core/pythagorean_triple.bril	81.65	40.49	23.28	10.44	34.75	15.66	2.098×
28	bril/core/quadratic.bril	66.05	26.66	19.10	8.41	49.45	20.74	2.411×
29	bril/core/recfact.bril	37.05	11.80	7.17	2.34	9.88	3.52	3.062×
30	bril/core/relative-primes.bril	54.44	21.46	59.86	22.89	37.59	21.31	2.313×
31	bril/core/reverse.bril	65.14	27.28	72.97	31.82	43.01	18.88	2.323×
32	bril/core/sum-bits.bril	28.35	9.66	13.79	5.14	44.13	18.18	2.616×
33	bril/core/sum-check.bril	50.54	20.70	56.62	22.60	25.78	9.24	2.530×
34	bril/core/sum-divisors.bril	101.79	42.91	27.65	15.19	74.86	39.97	2.083×
35	bril/core/sum-sq-diff.bril	50.36	22.74	65.99	31.42	24.65	9.58	2.212×
36	bril/core/totient.bril	104.20	42.03	104.91	45.61	43.90	22.69	2.293×
37	bril/core/up-arrow.bril	42.46	15.88	88.21	38.68	106.79	55.45	2.159×
38	bril/float/conjugate-gradient.bril	253.11	137.87	548.99	320.33	470.82	251.85	1.793×
39	bril/float/cordic.bril	434.84	180.16	384.79	198.56	862.15	396.13	2.170×
40	bril/float/euler.bril	27.34	9.58	46.59	17.92	16.68	7.75	2.570×
41	bril/float/leibniz.bril	75.92	28.04	77.12	31.41	31.72	13.92	2.518×
42	bril/float/mandelbrot.bril	136.88	58.10	232.98	126.44	111.02	62.24	1.949×
43	bril/float/n_root.bril	67.00	29.58	99.75	49.98	72.63	36.05	2.071×
44	bril/float/newton.bril	34.37	12.90	57.36	21.85	43.55	19.45	2.496×
45	bril/float/norm.bril	100.32	49.23	154.06	94.22	129.62	77.51	1.738×
46	bril/float/ray-sphere-intersection.bril	16.06	5.86	15.79	5.95	23.10	13.32	2.187×
47	bril/float/riemann.bril	49.67	23.37	22.49	12.41	80.06	49.65	1.782×

APPENDIX B / FROZEN REPLAY

Per-workload inference

Median milliseconds from three fresh processes per backend. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
48	bril/float/sqrt.bril	65.84	29.84	68.79	29.70	40.06	18.44	2.240×
49	bril/long/dead-branch.bril	53.79	19.64	54.26	20.64	28.23	9.11	2.759×
50	bril/mem/adj2csr.bril	162.39	76.20	291.91	172.92	147.51	87.02	1.790×
51	bril/mem/adler32.bril	87.45	40.29	33.54	17.42	111.33	50.04	2.156×
52	bril/mem/binary-search.bril	53.01	22.37	81.47	32.80	63.61	31.09	2.297×
53	bril/mem/bubblesort.bril	96.41	43.07	165.97	93.05	102.12	45.16	2.011×
54	bril/mem/csrmv.bril	315.68	145.49	611.87	333.71	595.04	252.11	2.082×
55	bril/mem/fib.bril	43.42	15.07	93.75	40.00	35.53	12.27	2.564×
56	bril/mem/major-elm.bril	59.21	22.66	35.65	15.16	105.92	45.27	2.416×
57	bril/mem/mat-mul.bril	141.69	62.23	257.73	138.05	145.03	73.79	1.987×
58	bril/mem/max-subarray.bril	67.81	24.75	125.76	51.24	70.34	31.76	2.449×
59	bril/mem/quickselect.bril	100.49	46.40	168.59	84.54	170.24	89.85	1.990×
60	bril/mem/quicksort.bril	140.34	50.49	172.17	88.35	356.73	189.02	2.041×
61	bril/mem/vsmul.bril	46.15	17.83	87.58	35.50	51.50	23.57	2.409×
62	bril/mixed/cholesky_decomp.bril	366.41	164.19	549.21	342.52	438.56	266.18	1.752×
63	bril/mixed/mat-inv.bril	124.09	51.14	132.99	50.06	175.09	77.11	2.424×
64	fenwick/fenwick_tree.rs	137.97	65.84	201.22	121.06	108.05	43.94	1.937×
65	polybench/datamining/correlation.rs	1,161.59	510.12	2,595.91	1,485.50	1,021.38	507.43	1.909×
66	polybench/datamining/covariance.rs	725.03	384.67	1,955.14	1,057.96	625.97	333.47	1.861×
67	polybench/LA/blas/gemm.bril	707.08	303.10	1,323.96	789.68	529.51	292.62	1.848×
68	polybench/LA/blas/gemver.bril	873.51	362.90	1,961.79	1,209.65	703.09	299.06	1.891×
69	polybench/LA/blas/gesummv.bril	327.74	131.07	644.42	317.85	286.48	138.07	2.144×
70	polybench/LA/blas/symm.bril	592.18	245.10	1,276.47	599.04	557.17	250.35	2.216×
71	polybench/LA/blas/syr2k.bril	525.40	215.21	1,065.68	568.61	448.32	219.73	2.032×

APPENDIX B / FROZEN REPLAY

Per-workload inference

Median milliseconds from three fresh processes per backend. Within each pass, N denotes native egglog on unmarked flat input and C denotes the standalone Cell executor. The final column divides the three-pass sums. LA abbreviates *linear-algebra* in the paths.

ID	eggcc passing workload	P1 N	P1 C	P2 N	P2 C	P3 N	P3 C	N/C
72	polybench/LA/blas/syrk.bril	445.71	192.30	779.25	468.34	369.32	183.90	1.888×
73	polybench/LA/blas/trmm.bril	387.89	159.73	667.90	371.25	306.54	154.23	1.988×
74	polybench/LA/kernels/2mm.bril	1,319.62	504.05	2,065.08	1,357.63	979.02	500.01	1.848×
75	polybench/LA/kernels/3mm.bril	1,795.75	697.90	2,598.53	1,752.94	1,148.08	632.91	1.797×
76	polybench/LA/kernels/atax.bril	414.26	180.88	805.77	484.61	303.24	165.56	1.833×
77	polybench/LA/kernels/bicg.bril	387.31	170.80	810.63	437.70	473.34	206.81	2.050×
78	polybench/LA/kernels/doitgen.bril	664.15	303.82	1,170.41	639.85	537.02	261.03	1.969×
79	polybench/LA/kernels/mvt.bril	379.50	171.32	827.93	425.60	318.54	158.74	2.019×
80	polybench/LA/solvers/cholesky.bril	590.42	284.14	943.24	528.86	587.86	306.71	1.895×
81	polybench/LA/solvers/durbin.bril	256.09	117.77	542.45	303.38	202.66	98.16	1.928×
82	polybench/LA/solvers/gramschmidt.bril	655.55	291.62	1,489.68	736.48	561.75	267.66	2.089×
83	polybench/LA/solvers/lu.bril	508.77	265.48	813.67	483.65	489.84	275.93	1.768×
84	polybench/LA/solvers/ludcmp.bril	934.85	446.70	1,512.76	917.93	888.22	440.14	1.848×
85	polybench/LA/solvers/trisolv.bril	189.05	81.86	415.50	238.98	210.45	102.34	1.926×
86	polybench/medley/deriche.rs	2,228.65	836.91	4,705.78	2,595.57	2,016.06	996.88	2.021×
87	polybench/medley/floyd-warshall.rs	251.54	127.91	688.98	405.02	443.25	228.66	1.817×
88	polybench/medley/nussinov.rs	452.15	215.09	845.47	487.28	704.00	311.60	1.974×
89	polybench/stencils/adi.rs	793.57	376.87	2,626.90	1,191.77	740.80	372.00	2.144×
90	polybench/stencils/fdtd-2d.rs	1,262.78	643.04	3,616.57	2,150.94	1,078.94	574.05	1.769×
91	polybench/stencils/heat-3d.rs	1,323.87	538.56	4,044.35	1,633.45	1,122.11	449.31	2.476×
92	polybench/stencils/jacobi-1d.rs	174.18	88.34	457.25	273.59	146.81	82.22	1.752×
93	polybench/stencils/jacobi-2d.rs	484.27	237.03	1,641.41	760.59	418.15	210.18	2.106×
94	polybench/stencils/seidel-2d.rs	354.49	173.58	1,159.03	494.73	326.95	155.86	2.233×
95	raytrace/raytrace.rs	4,756.08	2,034.71	12,510.35	4,739.87	23,790.95	6,966.66	2.988×

MAINTAINER BRIEF / 中文摘要

从逐 workload 的原生对照判断架构收益

本报告比较的是原生 egglog 执行原始、未加 marker 的源码，与 EggFuse 独立 Cell CPU 执行器编译并执行同一份输入。共 96 个 workload、288 个冻结的 pass 输入，每个输入双方各运行三个独立进程。附录 A 用四页图逐个列出全部 workload、双方耗时和比值；附录 B 给出每个 pass 的原始中位数。

速度收益是否普遍，而非由单个大案例主导？ 三段推理中位数相加，原生 egglog 共 145.504 秒，Cell 共 66.286 秒；96 个 workload 更快，0 个更慢。等权几何平均为 2.197×。Raytrace 占总节省时间的 34.5%。去掉它，按总时间仍为 1.988×。每个 pass 的输入均由原始 eggcc 生产运行提供；这些数值不代表两套后端分别驱动提取器后的完整编译时间。

如何把收益接进 egglog，而不是要求重写整个引擎？ 先在原生动作缓冲区前加入可选的编译入口和寄存器查询计划。共享入口向规则投递已解码参数，剩余查询继续使用 Free Join。所有匹配仍进入原生的有序动作缓冲区，保留 interning、merge、rebuild 与 extraction。第二步再引入跨 Run 的事实引用：生产者已提交的结果只能在消费者被原调度激活时，经过可见性、当前等价类和完整键检查后复用。

内存、准备成本和正确性必须一起验收。 本轮全套对照使用普通 release、单线程，不用 PGO。离线准备共 826.63 秒，单次编译的总墙钟成本仍需单独评估。第 6 节的原生／Cell RSS 来自这套完整配置；第 9 节的回收实验另有配对控制，不把它的低内存与本轮最快时间拼成一个配置。

288 个新 Cell 终态与历史 Cell 终态一致，但这只证明 Cell 内部结果稳定。sum-bits 两端虽然都有 3,206 个投影行，具体 TermSingle 成员查询仍存在差异。上游试验应定位首次分歧，比较有序 RHS、可见事实、函数值、等价类和提取程序行为。本报告的证据支持一次可逐步验收的架构试验；正式替换后端还需要通过这一语义门槛。